Chapter 7 ▪ Internationalization

There's a big world out there; we hope that lots of its inhabitants will be interested in your software. The Internet, after all, effortlessly spans the barriers between countries. On the other hand, when you pay no attention to an international audience, *you* are putting up a barrier.

The Java programming language was the first language designed from the ground up to support internationalization. From the beginning, it had the one essential feature needed for effective internationalization: It used Unicode for all strings. Unicode support makes it easy to write Java programs that manipulate strings in most of the world's languages.

Many programmers believe that all they need to do to internationalize their application is to support Unicode and to translate the messages in the user interface. However, as this chapter demonstrates, there is a lot more to internationalizing programs than just Unicode support. Dates, times, currencies, even numbers are formatted differently in different parts of the world. You need an easy way to configure menu and button names, message strings, and keyboard shortcuts for different languages.

In this chapter, you will see how to write internationalized Java programs and how to localize dates, times, numbers, text, and GUIs. I will show you the tools that Java offers for writing internationalized programs. The chapter concludes with a complete example—a retirement calculator with a user interface in English, German, and Chinese.

7.1. Locales

When you look at an application that is adapted to an international market, the most obvious difference you notice is the language. This observation is actually a bit too limiting for true internationalization, since countries can share a common language, but you might still need to do some work to make computer users of both countries happy. As Oscar Wilde famously said: "We have really everything in common with America nowadays, except, of course, language."

7.1.1. Why Locales?

When you provide international versions of a program, all program messages need to be translated to the local language. However, simply translating the user interface text is not sufficient. There are many more subtle differences—for example, numbers are formatted quite differently in English and German. The number

123,456.78

should be displayed as

123.456,78

for a German user—that is, the roles of the decimal point and the decimal comma separator are reversed. There are similar variations in the display of dates. In the United States, dates are (somewhat irrationally) displayed as month/day/year. Germany uses the more sensible order of day/month/year, whereas in China, the usage is year/month/day. Thus, the date

3/22/61

should be presented as

22.03.1961

to a German user. Of course, if the month names are written out explicitly, the difference in languages becomes apparent. The English

March 22, 1961

should be presented as

22. März 1961

in German, or

1961 年 3 月 22 日

in Chinese.

A *locale* captures local preferences such as these. Whenever you present numbers, dates, currency values, and other items whose formatting varies by language or location, you need to use locale-aware APIs.

7.1.2. Specifying Locales

A locale is made up of up to five components:

1. A language, specified by two or three lowercase letters, such as `en` (English), `de` (German), or `zh` (Chinese). **Table 7.1** shows common codes.
2. Optionally, a script, specified by four letters with an initial uppercase, such as `Latn` (Latin), `Cyrl` (Cyrillic), or `Hant` (traditional Chinese characters). This can be useful because some languages, such as Serbian, are written in Latin or Cyrillic, and some Chinese readers prefer the traditional over the simplified characters.
3. Optionally, a country or region, specified by two uppercase letters or three digits, such as `US` (United States) or `CH` (Switzerland). **Table 7.2** shows common codes.
4. Optionally, a variant, specifying miscellaneous features such as dialects or spelling rules. Variants are rarely used nowadays. There used to be a "Nynorsk" variant of Norwegian, but it is now expressed with a different language code, `nn`. What used to be variants for the Japanese imperial calendar and Thai numerals are now expressed as extensions (see the next item).
5. Optionally, an extension. Extensions describe local preferences for calendars (such as the Japanese calendar), numbers (Thai instead of Western digits), and so on. The Unicode standard specifies some of these extensions. Extensions start with `u-` and a two-letter code specifying whether the extension deals with the calendar (`ca`), numbers (`nu`), and so on. For example, the extension `u-nu-thai` denotes the use of Thai numerals. Other extensions are entirely arbitrary and start with `x-`, such as `x-java`.

Rules for locales are formulated in the "Best Current Practices" memo BCP 47 of the Internet Engineering Task Force (**https://www.rfc-editor.org/info/bcp47**). You can find a more accessible summary at **https://www.w3.org/International/articles/language-tags**.

Table 7.1: Common ISO 639-1 Language Codes

Language	Code	Language	Code

Language	Code	Language	Code
Chinese	`zh`	Italian	`it`
Danish	`da`	Japanese	`ja`
Dutch	`nl`	Korean	`ko`
English	`en`	Norwegian	`no`
French	`fr`	Portuguese	`pt`
Finnish	`fi`	Spanish	`es`
German	`de`	Swedish	`sv`
Greek	`el`	Turkish	`tr`

Table 7.2: Common ISO 3166-1 Country Codes

Country	Code	Country	Code
Austria	`AT`	Japan	`JP`
Belgium	`BE`	Korea	`KR`
Canada	`CA`	The Netherlands	`NL`
China	`CN`	Norway	`NO`
Denmark	`DK`	Portugal	`PT`
Finland	`FI`	Spain	`ES`
Germany	`DE`	Sweden	`SE`
Great Britain	`GB`	Switzerland	`CH`
Greece	`GR`	Taiwan	`TW`
Ireland	`IE`	Turkey	`TR`
Italy	`IT`	United States	`US`

The codes for languages and countries seem a bit random because some of them are derived from local languages. German in German is Deutsch, Chinese in Chinese is zhongwen: hence `de` and `zh`. And Switzerland is `CH`, deriving from the Latin term *Confoederatio Helvetica* for the Swiss confederation.

Locales are described by tags—hyphenated strings of locale elements such as `en-US`.

In Germany, you would use a locale `de-DE`. Switzerland has four official languages (German, French, Italian, and Rhaeto-Romance). A German speaker in Switzerland would want to use a locale `de-CH`. This locale uses the rules for the German language, but currency values are expressed in Swiss francs, not euros.

If you only specify the language, say, `de`, then the locale cannot be used for country-specific issues such as currencies.

You can construct a `Locale` object from a tag string like this:

```
Locale usEnglish = Locale.forLanguageTag("en-US");
```

In the common case where you only specify a language, or language and country, you can use the `Locale.of` factory method:

```
Locale german = Locale.of("de");
Locale swissGerman = Locale.of("de", "CH");
```

The `toLanguageTag` method yields the language tag for a given locale. For example, `Locale.US.toLanguageTag()` is the string `"en-US"`.

For your convenience, there are predefined locale objects for various countries:

```
Locale.CANADA
Locale.CANADA_FRENCH
Locale.CHINA
Locale.FRANCE
Locale.GERMANY
Locale.ITALY
Locale.JAPAN
Locale.KOREA
Locale.PRC
Locale.TAIWAN
Locale.UK
Locale.US
```

A number of predefined locales specify just a language without a location:

```
Locale.CHINESE
Locale.ENGLISH
Locale.FRENCH
Locale.GERMAN
Locale.ITALIAN
Locale.JAPANESE
Locale.KOREAN
Locale.SIMPLIFIED_CHINESE
Locale.TRADITIONAL_CHINESE
```

The `Locale.ROOT` has language-neutral rules that can be useful for machine-processed strings.

Finally, the static `getAvailableLocales` method returns an array of all locales known to the virtual machine. The `availableLocales` method returns a stream instead.



Note: You can get all language codes as `Locale.getISOLanguages()` and all country codes as `Locale.getISOCountries()`.

7.1.3. The Default Locale

The static `getDefault` method of the `Locale` class initially gets the default locale as stored by the local operating system. You can change the default Java locale by calling the `setDefault` method with a different locale.

Some operating systems allow the user to specify different locales for displayed messages and for formatting. For example, a French speaker living in the United States can have French menus but currency values in dollar.

To obtain these preferences, call

```
Locale displayLocale = Locale.getDefault(Locale.Category.DISPLAY);
Locale formatLocale = Locale.getDefault(Locale.Category.FORMAT);
```



Note: In UNIX, you can specify separate locales for numbers, currencies, and dates, by setting the `LC_NUMERIC`, `LC_MONETARY`, and `LC_TIME` environment variables. Java does not pay attention to these settings.



Tip: If you want to test a locale that just has language and country settings, you can supply them on the command line when you launch your program. For example, here we set the default locale to `de-CH`:

```
java -Duser.language=de -Duser.region=CH MyProgram
```

7.1.4. Display Names

Once you have a locale, what can you do with it? Initially, not much, as it turns out. The only useful methods in the `Locale` class are those for identifying the language and country codes. The most important one is `getDisplayName`. It returns a string describing the locale. This string does not contain the cryptic two-letter codes, but is in a form that can be presented to a user, such as

```
German (Switzerland)
```

Actually, there is a problem here. The display name is issued in the default locale. That might not be appropriate. If your user already selected German as the preferred language, you probably want to present the string in German. You can do just that by passing the German locale as an argument. The code

```
Locale locale = Locale.of("de", "CH");
IO.println(locale.getDisplayName(Locale.GERMAN));
```

prints

```
Deutsch (Schweiz)
```

This example shows why you need `Locale` objects. You feed them to locale-aware methods that produce text that is presented to users in different locations. You will see many examples of this in the following sections.



Caution: Even such mundane operations as turning a string into lowercase or uppercase can be locale-specific. For example, in the Turkish locale, the lowercase of the letter I is a dotless ı. Programs that tried to normalize strings by storing them in lowercase have mysteriously failed for Turkish customers where I and the dotted i don't have the same lowercase version. It is a good idea to always use the variants of `toUpperCase` and `toLowerCase` with a `Locale` parameter. For example, try out:

```
String command = "QUIT".toLowerCase(Locale.of("tr"));
    // "quıt" with a dotless ı
```

Of course, in Turkey, where `Locale.getDefault()` yields just that locale, `"QUIT".toLowerCase()` is not the same as `"quit"`.

If you want to normalize English language strings to lowercase, you should pass an English locale to the `toLowerCase` method. For language-neutral processing, use `Locale.ROOT`.



Note: You can explicitly set the locale for input and output operations.

- When reading numbers from a `Scanner`, you can set its locale with the `useLocale` method.
- The `String.format` and `PrintWriter.printf` methods have overloaded forms with a `Locale` parameter.

`java.util.Locale` 1.1

- `static Locale.of(String language)` 19
- `static Locale.of(String language, String country)` 19
  construct a locale with the given language and country.
- `static Locale forLanguageTag(String languageTag)` 7
  constructs a locale corresponding to the given language tag.
- `static Locale getDefault()`
  returns the default locale.
- `static void setDefault(Locale newLocale)`
  sets the default locale.
- `String getDisplayName()`
  returns a name describing the locale, expressed in the current locale.
- `String getDisplayName(Locale inLocale)`
  returns a name describing the locale, expressed in the given locale.
- `String getLanguage()`
  returns the language code—a lowercase two-letter ISO 639 code.
- `String getDisplayLanguage()`
  returns the name of the language, expressed in the current locale.
- `String getDisplayLanguage(Locale inLocale)`
  returns the name of the language, expressed in the given locale.
- `String getCountry()`
  returns the country code as an uppercase two-letter ISO 3166 code.
- `static String[] getISOCountries()`
- `static Set<String> getISOCountries(Locale.IsoCountryCode type)` 9
  get all two-letter country codes, or all country codes with 2, 3, or 4 letters. The type is one of the enumeration constants `PART1_ALPHA2`, `PART1_ALPHA3`, and `PART3`.

- `String getDisplayCountry()`
  returns the name of the country, expressed in the current locale.
- `String getDisplayCountry(Locale inLocale)`
  returns the name of the country, expressed in the given locale.
- `String toLanguageTag()` 7
  returns the IETF BCP 47 language tag for this locale—for example, `"de-CH"`.
- `String toString()`
  returns a description of the locale, with the language and country separated by underscores (e.g., `"de_CH"`). Use this method only for debugging.

7.2. Number Formats

I already mentioned how number and currency formatting is highly locale-dependent. The Java library supplies a collection of formatter objects that can format and parse numeric values in the `java.text` package.

7.2.1. Formatting Numeric Values

Go through the following steps to format a number for a particular locale:

1. Get the locale object, as described in the preceding section.
2. Use a "factory method" to obtain a formatter object.
3. Use the formatter object for formatting and parsing.

The factory methods are static methods of the `NumberFormat` class with a `Locale` parameter. There are three factory methods: `getNumberInstance`, `getCurrencyInstance`, and `getPercentInstance`. These methods return objects that can format and parse numbers, currency amounts, and percentages, respectively. For example, here is how you can format a currency value in German:

```
Locale locale = Locale.GERMAN;
NumberFormat formatter = NumberFormat.getCurrencyInstance(locale);
double amount = 123456.78;
String result = formatter.format(amount);
```

The result is

123.456,78 €

Note that the currency symbol is € and that it is placed at the end of the string. Also, note the reversal of decimal points and decimal commas.

Java 12 adds a "compact" format with two styles: short (123K) and long (123 thousand). Here is how to obtain the short style:

```
formatter = NumberFormat.getCompactNumberInstance(locale, NumberFormat.Style.SHORT);
```

Conversely, to read in a number that was entered or stored with the conventions of a certain locale, use the `parse` method. For example, the following code parses the value that the user typed into a text field. The `parse` method can deal with decimal points and commas, as well as digits in other languages.

```
TextField inputField;
. . .
NumberFormat formatter = NumberFormat.getNumberInstance();
// get the number formatter for default locale
Number input = formatter.parse(inputField.getText().strip());
double x = input.doubleValue();
```

The return type of `parse` is the abstract type `Number`. The returned object is either a `Double` or a `Long` wrapper object, depending on whether the parsed number was a floating-point number. If you don't care about the distinction, you can simply use the `doubleValue` method of the `Number` class to retrieve the wrapped number.



Caution: Objects of type `Number` are not automatically unboxed—you cannot simply assign a `Number` object to a primitive type. Instead, use the `doubleValue` or `intValue` method.

If the text for the number is not in the correct form, the method throws a `ParseException`. For example, leading whitespace in the string is *not* allowed. (Call `strip` to remove it.) However, any characters that follow the number in the string are simply ignored, so no exception is thrown.

Note that the instances returned by the `getXxxInstance` factory methods are not actually of type `NumberFormat`. The `NumberFormat` type is an abstract class, and the actual formatters belong to one of its subclasses. The factory methods merely know how to locate the object that belongs to a particular locale.

You can get a list of the currently supported locales with the static `getAvailableLocales` method. That method returns an array of the locales for which number formatter objects can be obtained.



Note: You can use a `Scanner` to read localized integers and floating-point numbers. Call the `useLocale` method to set the locale.

The book companion code contains a GUI program that lets you experiment with number formatters (see Figure 7.1). The combo box at the top of the figure contains all locales with number formatters. You can choose between number, currency, and percentage formatters. Each time you make another choice, the number in the text field is reformatted. If you go through a few locales, you can get a good impression of the many ways that a number or currency value can be formatted. You can also type a different number and click the Parse button to call the `parse` method, which tries to parse what you entered. If your input is successfully parsed, it is passed to `format` and the result is displayed. If parsing fails, a "Parse error" message is displayed in the text field.



Figure 7.1: The `NumberFormatDemo` program

Listing 7.1 shows a text version of the number format explorer that is a bit easier to follow.

Listing 7.1 v2ch07/numberFormat/NumberFormatDemo2.java

```
1  package v2ch07.numberFormat;
2
3  import module java.base;
4  import v2ch07.util.Choices;
```

```
5
6  /**
7   * This program demonstrates formatting numbers under various locales.
8   */
9  class NumberFormatDemo2 {
10     void main() {
11         var locales = (Locale[]) NumberFormat.getAvailableLocales().clone();
12         Arrays.sort(locales, Comparator.comparing(Locale::getDisplayName));
13         Locale locale = Choices.choose(locales, Locale::getDisplayName);
14
15         var formatters = new LinkedHashMap<NumberFormat, String>();
16         formatters.put(NumberFormat.getNumberInstance(locale), "Number");
17         formatters.put(NumberFormat.getCompactNumberInstance(locale, NumberFormat.Style.SHORT),
18                 "Compact Short");
19         formatters.put(NumberFormat.getCompactNumberInstance(locale, NumberFormat.Style.LONG),
20                 "Compact Long");
21         formatters.put(NumberFormat.getPercentInstance(locale), "Percent");
22         formatters.put(NumberFormat.getCurrencyInstance(locale), "Currency");
23         NumberFormat formatter = Choices.choose(formatters);
24
25         String operation = Choices.choose("Format", "Parse");
26         if (operation.equals("Format")) {
27             double number = Double.parseDouble(
28                     IO.readln("Enter a floating-point number to format: "));
29             System.out.println(formatter.format(number));
30         }
31         else {
32             String text = IO.readln("Enter a floating-point number to parse: ");
33             try {
34                 System.out.println(formatter.parse(text));
35             }
36             catch (ParseException e) {
37                 System.out.println("ParseException " + e.getMessage());
38             }
39         }
40     }
41 }
```

`java.text.NumberFormat` 1.1

- `static Locale[] getAvailableLocales()`
  returns an array of `Locale` objects for which `NumberFormat` formatters are available.
- `static NumberFormat getNumberInstance()`
- `static NumberFormat getNumberInstance(Locale inLocale)`
- `static NumberFormat getCurrencyInstance()`
- `static NumberFormat getCurrencyInstance(Locale inLocale)`
- `static NumberFormat getPercentInstance()`
- `static NumberFormat getPercentInstance(Locale inLocale)`
  return a formatter for numbers, currency amounts, or percentage values for the current locale or for the given locale.

- `static NumberFormat getCompactNumberInstance()` [12]
- `static NumberFormat getCompactNumberInstance(Locale locale, NumberFormat.Style formatStyle)` [12]
  return a compact number formatter in the current locale and `SHORT` style, or in the given locale and style (either `SHORT` or `LONG`).
- `String format(double number)`
- `String format(long number)`
  return the string resulting from formatting the given floating-point number or integer.
- `Number parse(String source)`
  parses the given string and returns the number value, as a `Double` if the input string describes a floating-point number or as a `Long` otherwise. The beginning of the string must contain a number; no leading whitespace is allowed. The number can be followed by other characters, which are ignored. Throws `ParseException` if parsing was not successful.
- `void setParseIntegerOnly(boolean value)`
- `boolean isParseIntegerOnly()`
  set or get a flag to indicate whether this formatter should parse only integer values.
- `void setGroupingUsed(boolean newValue)`
- `boolean isGroupingUsed()`
  set or get a flag to indicate whether this formatter emits and recognizes decimal separators (such as `100,000`).
- `void setMinimumIntegerDigits(int newValue)`
- `int getMinimumIntegerDigits()`
- `void setMaximumIntegerDigits(int newValue)`
- `int getMaximumIntegerDigits()`
- `void setMinimumFractionDigits(int newValue)`
- `int getMinimumFractionDigits()`
- `void setMaximumFractionDigits(int newValue)`
- `int getMaximumFractionDigits()`
  set or get the maximum or minimum number of digits allowed in the integer or fractional part of a number.

7.2.2. The `DecimalFormat` Class

In most locales, the `NumberFormat` factory methods that you saw in the preceding section return an instance of the `DecimalFormat` class. This class describes the formatting variations that occur around the world. You can modify individual settings, or create an entirely new formatter. A pattern syntax makes this a bit more convenient.

The pattern describes the number of required and optional digits, and the prefixes and suffixes for positive and negative numbers. There are also a couple more esoteric settings—see **Table 7.3**.

Consider this example:

```
var formatter = new DecimalFormat("0.00;(#)");
```

The semicolon separates the positive and the optional negative part. From the positive part, we see that there must be at least one digit in the integer part and at least two digits in the fractional part. This holds for all numbers, regardless of sign. Negative numbers use the accounting format—a prefix `(` and a suffix `)`.

The prefix or suffix may contain a `¤` currency sign (U+00A4) to denote where the currency symbol should go.

If you need to insert a special character into the prefix or suffix, precede it with a single quote. For example, a prefix `'#` is a literal hash sign, and `o''clock` has one quotation mark.

Table 7.3: `DecimalFormat` Properties

Property Name	Description	Pattern
`groupingSize`	The number of integer digits grouped together (usually 3 or 4)	The number of digits between the last `,` and the end of the integer part, such as `#,###` for a grouping size of 3.
`minimum` / `maximumFractionDigits`	The minimum and maximum number of digits of the fractional part	Use required (`0`) and optional (`#`) digits in the fractional part: `.00##` .
`minimumIntegerDigits`	The minimum number of digits of the integer part	Use required (`0`) digits in the integer part: `000` .
`maximumIntegerDigits`	The maximum number of integer digits	With exponential notation, the number of `#` digits in the integer part, such as `###,##0.00E0` . Otherwise, cannot be set in pattern.
—	the minimum number of digits in the exponential part	Use `0` digits in the `E` part: `#.00E00` .
`multiplier`	The multiplier for percent, per thousand	Use `%` for 100, `‰` (U+2030) for 1000.
`positivePrefix` / `Suffix` , `negativePrefix` / `Suffix`	The prefix and suffix for positive or negative numbers	Surround the positive or negative part of the pattern with the literal prefix or suffix, e.g. `+#0.00;(#)` uses a `+` prefix and no suffix for positive numbers and encloses negative numbers in parentheses.
`decimalSeparatorAlwaysShown`	`true` if the separator is shown for a zero fractional part	Cannot be set in pattern.

The actual symbols for the digits, separators, and miscellaneous other number parts are taken from a `DecimalFormatSymbols` object that you can also customize. **Table 7.4** lists its properties.

Table 7.4: `DecimalFormatSymbol` Properties

Property	Type	Description
`currencySymbol`	`String`	A string such as `"$"` or `"EUR"` to use where the prefix or suffix has a `¤` currency sign
`decimalSeparator`, `monetaryDecimalSeparator`	`char`	The decimal separator to use with numbers or currency values
`exponentSeparator`	`String`	The string before the exponent part, usually `"E"`
`groupingSeparator`, `monetaryGroupingSeparator` (since Java 15)	`char`	The group separator to use with numbers or currency values
`infinity`, `naN`	`String`	The strings for formatting `Double.POSITIVE_INFINITY`, `Double.NEGATIVE_INFINITY`, and `Double.NaN`
`internationalCurrencySymbol`	String	The ISO 4217 currency symbol—see **Section 7.2.3**
`minusSign`	`char`	The minus sign that is used when there is no negative pattern
`percent`, `perMill`	`char`	The characters used for percent or per mille
`zeroDigit`	`char`	The character for the digit zero. The other digits are the subsequent nine Unicode characters.

Suppose, for example, that you want to show numbers in the US style, with `,` as group separator and `.` as decimal separator, no matter what the locale is. Customize the `DecimalFormatSymbols` and the formatter:

```
DecimalFormatSymbols symbols = new DecimalFormatSymbols(locale);
symbols.setGroupingSeparator(',');
symbols.setDecimalSeparator('.');
DecimalFormat formatter = (DecimalFormat) NumberFormat.getNumberInstance(locale);
formatter.setDecimalFormatSymbols(symbols);
```

7.2.3. Currencies

To format a currency value, you can use the `NumberFormat.getCurrencyInstance` method. However, that method is not very flexible—it returns a formatter for a single currency. Suppose you prepare an invoice for an American customer in which some amounts are in dollars and others are in euros. You can't just use two formatters:

```
NumberFormat dollarFormatter = NumberFormat.getCurrencyInstance(Locale.US);
NumberFormat euroFormatter = NumberFormat.getCurrencyInstance(Locale.GERMANY);
```

Your invoice would look very strange, with some values shown as $100,000 and others as 100.000 €. (Note that the euro value uses a decimal point, not a comma.)

Instead, use the `Currency` class to control the currency used by the formatters. You can get a `Currency` object by passing a currency identifier to the static `Currency.getInstance` method. Then call the `setCurrency` method for each formatter. Here is how you would set up the euro formatter for your American customer:

```
NumberFormat euroFormatter = NumberFormat.getCurrencyInstance(Locale.US);
euroFormatter.setCurrency(Currency.getInstance("EUR"));
```

The currency identifiers are defined by ISO 4217 (see **https://www.iso.org/iso-4217-currency-codes.html**). **Table 7.5** provides a partial list.

Table 7.5: Currency Identifiers

Currency Value	Identifier	Numeric Code
U.S. Dollar	USD	840
Euro	EUR	978
British Pound	GBP	826
Japanese Yen	JPY	392
Chinese Renminbi (Yuan)	CNY	156
Indian Rupee	INR	356
Russian Ruble	RUB	643

`java.util.Currency` 1.4

- `static Currency getInstance(String currencyCode)`
- `static Currency getInstance(Locale locale)`
  return the `Currency` instance for the given ISO 4217 currency code or the country of the given locale.
- `String toString()`
- `String getCurrencyCode()`
- `String getNumericCode()` 7
- `String getNumericCodeAsString()` 9
  get the ISO 4217 alphabetic or numeric currency code of this currency.
- `String getSymbol()`
- `String getSymbol(Locale locale)`
  get the formatting symbol of this currency for the default locale or the given locale. For example, the symbol for USD can be `"$"` or `"US$"`, depending on the locale.
- `int getDefaultFractionDigits()`
  gets the default number of fraction digits of this currency.
- `static Set<Currency> getAvailableCurrencies()` 7
  gets all available currencies.

7.3. Date and Time

When you are formatting date and time, you should be concerned with four locale-dependent issues:

- The names of months and weekdays should be presented in the local language.
- There will be local preferences for the order of year, month, and day.
- The Gregorian calendar might not be the local preference for expressing dates.
- The time zone of the location must be taken into account.

The `DateTimeFormatter` class from the `java.time` package handles these issues. Pick one of the formatting styles shown in **Table 7.6**. Then, get a formatter:

```
FormatStyle style = . . .; // One of FormatStyle.SHORT, FormatStyle.MEDIUM, . . .
DateTimeFormatter dateFormatter = DateTimeFormatter.ofLocalizedDate(style);
DateTimeFormatter timeFormatter = DateTimeFormatter.ofLocalizedTime(style);
DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofLocalizedDateTime(style);
    // or DateTimeFormatter.ofLocalizedDateTime(dateStyle, timeStyle)
```

These formatters use the current locale. To use a different locale, use the `withLocale` method:

```
DateTimeFormatter dateFormatter =
    DateTimeFormatter.ofLocalizedDate(style).withLocale(locale);
```

Now you can format a `LocalDate`, `LocalDateTime`, `LocalTime`, or `ZonedDateTime`:

```
ZonedDateTime appointment = . . .;
String formatted = formatter.format(appointment);
```

Table 7.6: Date and Time Formatting Styles

Style	Date	Time
`SHORT`	`7/16/69`	`9:32 AM`
`MEDIUM`	`Jul 16, 1969`	`9:32:00 AM`
`LONG`	`July 16, 1969`	`9:32:00 AM EDT` in `en-US`, `9:32:00 MSZ` in `de-DE` (only for `ZonedDateTime`)
`FULL`	`Wednesday, July 16, 1969`	`9:32:00 AM EDT` in `en-US`, `9:32 Uhr MSZ` in `de-DE` (only for `ZonedDateTime`)



Note: Here we use the `DateTimeFormatter` class from the `java.time` package. There is also a legacy `java.text.DateFormatter` class that works with `Date` and `Calendar` objects.

You can use one of the static `parse` methods of `LocalDate`, `LocalDateTime`, `LocalTime`, or `ZonedDateTime` to parse a date or time in a string:

```
LocalTime time = LocalTime.parse("9:32 AM", formatter);
```

These methods are not suitable for parsing human input, at least not without preprocessing. For example, the short time formatter for the United States will parse `"9:32 AM"` but not `"9:32AM"` or `"9:32 am"`.



Caution: Date formatters parse nonexistent dates, such as November 31, and adjust them to the last date in the given month.

Sometimes, you need to display just the names of weekdays and months, for example, in a calendar application. Call the `getDisplayName` method of the `DayOfWeek` and `Month` enumerations.

```
for (Month m : Month.values())
    IO.println(m.getDisplayName(textStyle, locale));
```

Table 7.7 shows the text styles. The `STANDALONE` versions are for display outside a formatted date. For example, in Finnish, January is “tammikuuta” inside a date, but “tammikuu” standalone.

Table 7.7: Values of the `java.time.format.TextStyle` Enumeration

Style	Example
`FULL` / `FULL_STANDALONE`	January
`SHORT` / `SHORT_STANDALONE`	Jan
`NARROW` / `NARROW_STANDALONE`	J



Note: The first day of the week can be Saturday, Sunday, or Monday, depending on the locale. You can obtain it like this:

```
DayOfWeek first = WeekFields.of(locale).getFirstDayOfWeek();
```

Listing 7.2 shows the `DateFormat` class in action. You can select a locale and see how the date and time are formatted in different places around the world.

Figure 7.2 shows a GUI version of the program, which is included in the book companion code. You may need to install Chinese fonts to see the Chinese characters displayed.

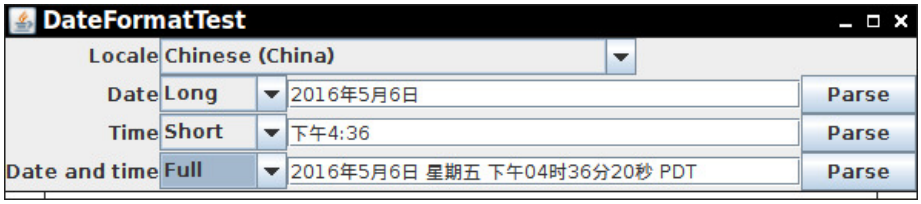


Figure 7.2: The `DateFormatDemo` program

Listing 7.2 v2ch07/dateFormat/DateTimeFormatDemo2.java

```
1  package v2ch07.dateFormat;
2
3  import module java.base;
4  import v2ch07.util.*;
5
6  /**
7   * This program demonstrates formatting dates under various locales.
8   */
9  class DateTimeFormatDemo2 {
10     void main() {
11         var locales = (Locale[]) NumberFormat.getAvailableLocales().clone();
12         Arrays.sort(locales, Comparator.comparing(Locale::getDisplayName));
13         Locale locale = Choices.choose(locales, Locale::getDisplayName);
14         FormatStyle style = Choices.choose(FormatStyle.class,
15                 "Short", "Medium", "Long", "Full");
16         String type = Choices.choose("Date", "Time", "Date and Time");
17
18         if (type.equals("Date")) {
19             DateTimeFormatter formatter
20                 = DateTimeFormatter.ofLocalizedDate(style).withLocale(locale);
21             System.out.println(formatter.format(LocalDate.now()));
22             String input = IO.readln("Enter another date: ");
23             LocalDate date = LocalDate.parse(input, formatter);
24             System.out.println(formatter.format(date));
25         } else if (type.equals("Time")) {
26             DateTimeFormatter formatter
27                 = DateTimeFormatter.ofLocalizedTime(style).withLocale(locale);
28             System.out.println(formatter.format(LocalTime.now()));
29             String input = IO.readln("Enter another time: ");
30             LocalTime time = LocalTime.parse(input, formatter);
31             System.out.println(formatter.format(time));
32         } else {
33             DateTimeFormatter formatter
34                 = DateTimeFormatter.ofLocalizedDateTime(style).withLocale(locale);
35             System.out.println(formatter.format(ZonedDateTime.now()));
36             String input = IO.readln("Enter another date and time: ");
37             ZonedDateTime dateTime = ZonedDateTime.parse(input, formatter);
38             System.out.println(formatter.format(dateTime));
39         }
40     }
41 }
```

`java.time.format.DateTimeFormatter` 8

- `static DateTimeFormatter ofLocalizedDate(FormatStyle dateStyle)`
- `static DateTimeFormatter ofLocalizedTime(FormatStyle dateStyle)`
- `static DateTimeFormatter ofLocalizedDateTime(FormatStyle dateTimeStyle)`
- `static DateTimeFormatter ofLocalizedDateTime(FormatStyle dateStyle, FormatStyle timeStyle)`
  return `DateTimeFormatter` instances that format dates, times, or dates and times with the specified styles.
- `DateTimeFormatter withLocale(Locale locale)`
  returns a copy of this formatter with the given locale.

- `String format(TemporalAccessor temporal)`
  returns the string resulting from formatting the given date/time.

`java.time.LocalDate` 8

`java.time.LocalTime` 8

`java.time.LocalDateTime` 8

`java.time.ZonedDateTime` 8

- `static Xxx parse(CharSequence text, DateTimeFormatter formatter)`
  parses the given string and returns the `LocalDate`, `LocalTime`, `LocalDateTime`, or `ZonedDateTime` described in it. Throws a `DateTimeParseException` if parsing was not successful.

7.4. Collation and Normalization

Most programmers know how to compare strings with the `compareTo` method of the `String` class. Unfortunately, when interacting with human users, this method is not very useful. The `compareTo` method uses the values of the UTF-16 encoding of the string, which leads to absurd results, even in English. For example, the following five strings are ordered according to the `compareTo` method:

```
America
Zulu
able
zebra
Ångström
```

For dictionary ordering, you would want to consider upper case and lower case to be equivalent. To an English speaker, the sample list of words would be ordered as

```
able
America
Ångström
zebra
Zulu
```

However, that order would not be acceptable to a Swedish user. In Swedish, the letter Å is different from the letter A, and it is collated *after* the letter Z! That is, a Swedish user would want the words to be sorted as

```
able
America
zebra
Zulu
Ångström
```

To obtain a locale-sensitive comparator, call the static `Collator.getInstance` method:

```
Collator coll = Collator.getInstance(locale);
words.sort(coll); // Collator implements Comparator<Object>
```



Note: Since Java 21, you can specify unusual collators with an extension. For example, the locale with language tag `sv-u-co-trad` uses traditional Swedish collation, which does not distinguish between "v" and "w".

Since the `Collator` class implements the `Comparator` interface, you can pass a `Collator` object to the `List.sort(Comparator)` method to sort a list of strings.

There are a couple of advanced settings for collators. You can set a collator's *strength* to adjust how selective it should be. Character differences are classified as *primary*, *secondary*, or *tertiary*. For example, in English, the difference between "A" and "Z" is considered primary, the difference between "A" and "Å" is secondary, and between "A" and "a" is tertiary.

By setting the strength of the collator to `Collator.PRIMARY`, you tell it to pay attention only to primary differences. By setting the strength to `Collator.SECONDARY`, you instruct the collator to take secondary differences into account. That is, two strings will be more likely to be considered different when the strength is set to "secondary" or "tertiary," as shown in **Table 7.8**.

Table 7.8: Collations with Different Strengths (English Locale)

Primary	Secondary	Tertiary
Angstrom = Ångström	Angstrom ≠ Ångström	Angstrom ≠ Ångström
Able = able	Able = able	Able ≠ able

When the strength has been set to `Collator.IDENTICAL`, no differences are allowed. This setting is mainly useful in conjunction with a rather technical collator setting, the *decomposition mode*, which we take up next.

Occasionally, a character or sequence of characters can be described in more than one way in Unicode. For example, an "Å" can be Unicode character U+00C5, or it can be expressed as a plain "A" (U+0065) followed by a "°" ("combining ring above"; U+030A). Perhaps more surprisingly, the letter sequence "ffi" can be described with a single character "Latin small ligature ffi" with code U+FB03. (One could argue that this is a presentation issue that should not have resulted in different Unicode characters, but we don't make the rules.)

The Unicode standard defines four *normalization forms* (D, KD, C, and KC) for strings. See **https://www.unicode.org/unicode/reports/tr15/tr15-23.html** for the details. In the normalization form C, accented characters are always composed. For example, a sequence of "A" and a combining ring above "°" is combined into a single character "Å". In form D, accented characters are always decomposed into their base letters and combining accents: "Å" is turned into "A" followed by "°". Forms KC and KD also decompose characters such as ligatures or the trademark symbol.

You can choose the degree of normalization that you want a collator to use. The value `Collator.NO_DECOMPOSITION` does not normalize strings at all. This option is faster, but it might not be appropriate for texts that express characters in multiple forms. The default, `Collator.CANONICAL_DECOMPOSITION`, uses the normalization form D. This is useful for texts that contain accents but not ligatures. Finally, "full decomposition" uses normalization form KD. See **Table 7.9** for examples.

Table 7.9: Differences between Decomposition Modes

No Decomposition	Canonical Decomposition	Full Decomposition
Å ≠ A°	Å = A°	Å = A°
™ ≠ TM	™ ≠ TM	™ = TM

It is wasteful to have the collator decompose a string many times. If one string is compared against other strings many times, you can save the decomposition in a *collation* key object. The `getCollationKey` method returns a `CollationKey` object that you can use for further, faster comparisons. Here is an example:

```
String a = . . .;
CollationKey aKey = coll.getCollationKey(a);
if(aKey.compareTo(coll.getCollationKey(b)) == 0) // fast comparison
    . . .
```

Finally, you might want to convert strings into their normalized forms even if you don't do collation—for example, to store strings in a database or communicate with another program. The `java.text.Normalizer` class carries out the normalization process. Here is an example:

```
String name = "Ångström";
String normalized = Normalizer.normalize(name, Normalizer.Form.NFD);
    // uses normalization form D
```

The normalized string contains ten characters. The "Å" and "ö" are replaced by "A°" and "o¨" sequences.

However, that is not usually the best form for storage and transmission. Normalization form C first applies decomposition and then combines the accents back in a standardized order. According to the W3C, this is the recommended mode for transferring data over the Internet.

The program in **Listing 7.3** lets you experiment with collation order. The book companion code also has a GUI version. Type a word into the text field and click the Add button to add it to the list of words. Each time you add another word, or change the locale, strength, or decomposition mode, the list of words is sorted again. An `=` sign indicates words that are considered identical (see **Figure 7.3**).

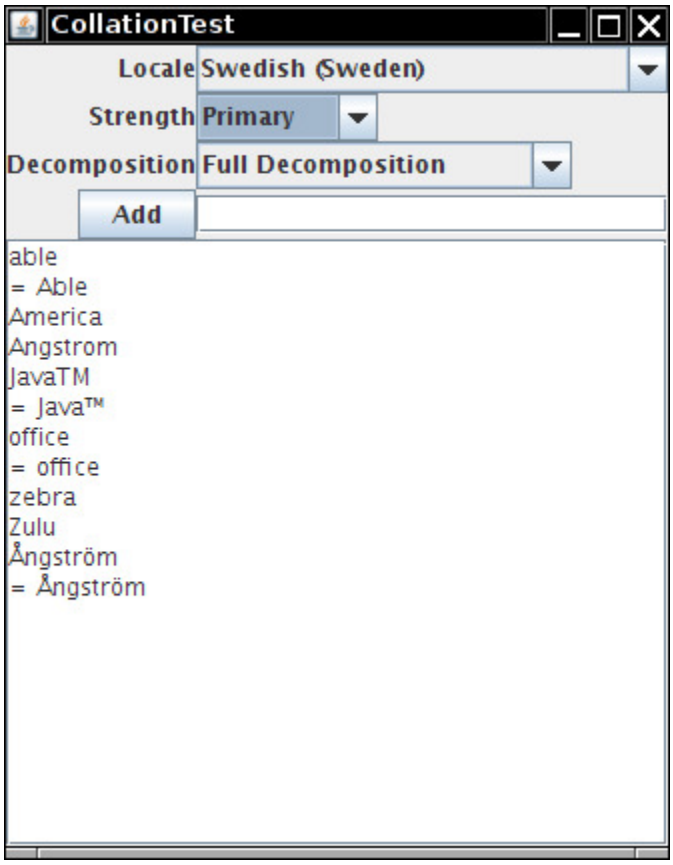


Figure 7.3: The `CollationDemo` program

The locale names are displayed in sorted order, using the collator of the default locale. If you run this program with the US English locale, note that “Norwegian (Norway,Nynorsk)” comes before “Norwegian (Norway)”. The standard string comparison would have reversed the order since the Unicode value of the comma character is greater than the Unicode value of the closing parenthesis.

Listing 7.3 v2ch07/collation/CollationDemo2.java

```
1  package v2ch07.collation;
2
3  import module java.base;
4  import v2ch07.util.Choices;
5
6  /**
7   * This program demonstrates collating strings under various locales.
8   */
9  class CollationDemo2 {
10    void main() {
11       var locales = (Locale[]) NumberFormat.getAvailableLocales().clone();
12       Arrays.sort(locales, Comparator.comparing(Locale::getDisplayName));
13       Locale locale = Choices.choose(locales, Locale::getDisplayName);
14
15       Collator collator = Collator.getInstance(locale);
16       int strength = Choices.choose(Collator.class,
17          "Primary", "Secondary", "Tertiary", "Identical");
18       int decomposition = Choices.choose(Collator.class,
19          "Canonical Decomposition", "Full Decomposition", "No Decomposition");
20
21       List<String> strings = new ArrayList<>();
22       strings.add("America");
23       strings.add("able");
24       strings.add("Zulu");
```

```
      strings.add("zebra");
      strings.add("\u00C5ngstr\u00F6m");
      strings.add("A\u030angstro\u0308m");
      strings.add("Angstrom");
      strings.add("Able");
      strings.add("office");
      strings.add("o\uFB03ce");
      strings.add("Java\u2122");
      strings.add("JavaTM");

      collator.setStrength(strength);
      collator.setDecomposition(decomposition);

      strings.sort(collator);

      for (int i = 0; i < strings.size(); i++) {
         String s = strings.get(i);
         if (i > 0 && collator.compare(s, strings.get(i - 1)) == 0)
            System.out.print("= ");
         System.out.println(s);
      }
   }
}
```

`java.text.Collator` 1.1

- `static Locale[] getAvailableLocales()`
  returns an array of `Locale` objects for which `Collator` objects are available.
- `static Collator getInstance()`
- `static Collator getInstance(Locale l)`
  return a collator for the default locale or the given locale.
- `int compare(String a, String b)`
  returns a negative value if `a` comes before `b`, `0` if they are considered identical, and a positive value otherwise.
- `boolean equals(String a, String b)`
  returns `true` if `a` and `b` are considered identical, `false` otherwise.
- `void setStrength(int strength)`
- `int getStrength()`
  set or get the strength of the collator. Stronger collators tell more words apart. Strength values are `Collator.PRIMARY`, `Collator.SECONDARY`, and `Collator.TERTIARY`.
- `void setDecomposition(int decomp)`
- `int getDecompositon()`
  set or get the decomposition mode of the collator. The more a collator decomposes a string, the more strict it will be in deciding whether two strings should be considered identical. Decomposition values are `Collator.NO_DECOMPOSITION`, `Collator.CANONICAL_DECOMPOSITION`, and `Collator.FULL_DECOMPOSITION`.
- `CollationKey getCollationKey(String a)`
  returns a collation key that contains a decomposition of the characters in a form that can be quickly compared against another collation key.

`java.text.CollationKey` 1.1

- `int compareTo(CollationKey b)`
  returns a negative value if this key comes before `b`, `0` if they are considered identical, and a positive value otherwise.

`java.text.Normalizer` 6

- `static String normalize(CharSequence str, Normalizer.Form form)`
  returns the normalized form of `str`. The `form` value is one of `NFD`, `NFKD`, `NFC`, or `NFKC`.

7.5. Message Formatting

The Java library has a `MessageFormat` class that formats text with variable parts. It is similar to formatting with the `printf` method, but it supports locales and formats for numbers and dates. We will examine that mechanism in the following sections.

7.5.1. Formatting Numbers and Dates

Here is a typical message format string:

```
"On {2}, a {0} destroyed {1} houses and caused {3} of damage."
```

The numbers in braces are placeholders for actual names and values. The static method `MessageFormat.format` lets you substitute values for the variables. It is a "varargs" method, so you can simply supply the parameters as follows:

```
String msg
    = MessageFormat.format("On {2}, a {0} destroyed {1} houses and caused {3} of damage.",
        "hurricane", 99, new GregorianCalendar(1999, 0, 1).getTime(), 10.0E8);
```

In this example, the placeholder `{0}` is replaced with `"hurricane"`, `{1}` is replaced with `99`, and so on.

The result of our example is the string

```
On 1/1/99 12:00 AM, a hurricane destroyed 99 houses and caused 100,000,000 of damage.
```

That is a start, but it is not perfect. We don't want to display the time "12:00 AM," and we want the damage amount printed as a currency value. The way we do this is by supplying an optional format for some of the placeholders:

```
"On {2,date,long}, a {0} destroyed {1} houses and caused {3,number,currency} of damage."
```

This example code prints:

```
On January 1, 1999, a hurricane destroyed 99 houses and caused $100,000,000 of damage.
```

In general, the placeholder index can be followed by a *type* and a *style*. Separate the index, type, and style by commas. The type can be any of

```
number
time
date
choice
```

If the type is `number`, then the style can be

```
integer
currency
percent
```

or it can be a `DecimalFormat` pattern such as `$,##0`—see **Section 7.2.2** for the possible formats.

If the type is either `time` or `date`, then the style can be

```
short
medium
long
full
```

or a date format pattern such as `yyyy-MM-dd`. (See the documentation of the `SimpleDateFormat` class for the possible formats.)



Caution: To include a brace in a message format string, surround it with single quotes, for example: `"A {0} is enclosed in '{' and '}' symbols".` It is best to use curly apostrophes for text: `"{0}'s favorite symbols are braces"`. But if you must have straight single quotes, you need to double them:

```
String template = "<a href=''{0}''>{1}</a>";
```

The static `MessageFormat.format` method uses the current locale to format the values. To format with an arbitrary locale, you have to work a bit harder because there is no "varargs" method that you can use. You need to place the values to be formatted into an `Object[]` array, like this:

```
var formatter = new MessageFormat(pattern, locale);
String message = formatter.format(new Object[] { values });
```

`java.text.MessageFormat` 1.1

- `MessageFormat(String pattern)`
- `MessageFormat(String pattern, Locale locale)`
  construct a message format object with the specified pattern and locale.
- `void applyPattern(String pattern)`
  sets the pattern of a message format object to the specified pattern.

- `void setLocale(Locale locale)`
- `Locale getLocale()`
  set or get the locale to be used for the placeholders in the message. The locale is *only* used for subsequent patterns that you set by calling the `applyPattern` method.
- `static String format(String pattern, Object... args)`
  formats the pattern string by using `args[i]` as input for placeholder `{i}`.
- `StringBuffer format(Object args, StringBuffer result, FieldPosition pos)`
  formats the pattern of this `MessageFormat`. The `args` parameter must be an array of objects. The formatted string is appended to `result`, and `result` is returned. If `pos` equals `new FieldPosition(MessageFormat.Field.ARGUMENT)`, its `beginIndex` and `endIndex` properties are set to the location of the text that replaces the `{1}` placeholder. Supply `null` if you are not interested in position information.

`java.text.Format` 1.1

- `String format(Object obj)`
  formats the given object, according to the rules of this formatter. This method calls `format(obj, new StringBuffer(), new FieldPosition(0)).toString()`.

7.5.2. Choice Formats

Let's look closer at the pattern of the preceding section:

```
"On {2}, a {0} destroyed {1} houses and caused {3} of damage."
```

If we replace the disaster placeholder `{0}` with `"earthquake"`, the sentence is not grammatically correct in English:

```
On January 1, 1999, a earthquake destroyed . . .
```

What we really want to do is integrate the article "a" into the placeholder:

```
"On {2}, {0} destroyed {1} houses and caused {3} of damage."
```

The `{0}` would then be replaced with `"a hurricane"` or `"an earthquake"`. That is especially appropriate if this message needs to be translated into a language where the gender of a word affects the article. For example, in German, the pattern would be

```
"{0} zerstörte am {2} {1} Häuser und richtete einen Schaden von {3} an."
```

The placeholder would then be replaced with the grammatically correct combination of article and noun, such as `"Ein Wirbelsturm"` or `"Eine Naturkatastrophe"`.

Now let us turn to the `{1}` parameter. If the disaster wasn't all that catastrophic, `{1}` might be replaced with the number 1, and the message would read:

```
On January 1, 1999, a mudslide destroyed 1 houses and . . .
```

Ideally, we would like the message to vary according to the placeholder value, so it would read

```
no houses
one house
2 houses
. . .
```

depending on the placeholder value. The `choice` formatting option was designed for this purpose.

A choice format is a sequence of pairs, each containing

- A *lower limit*
- A *format string*

The lower limit and format string are separated by a `#` character, and the pairs are separated by `|` characters.

For example,

```
{1,choice,0#no houses|1#one house|2#{1} houses}
```

Table 7.10 shows the effect of this format string for various values of `{1}`.

Table 7.10: String Formatted by Choice Format

{1}	Result	{1}	Result
0	"no houses"	3	"3 houses"
1	"one house"	-1	"no houses"

Why do we use `{1}` twice in the format string? When the message format applies the choice format to the `{1}` placeholder and the value is `2`, the choice format returns `"{1} houses"`. That string is then formatted again by the message format, and the answer is spliced into the result.



Note: This example shows that the designer of the choice format was a bit muddleheaded. If you have three format strings, you need two limits to separate them. In general, you need *one fewer limit* than you have format strings. As you saw in **Table 7.10**, the `MessageFormat` class ignores the first limit.

The syntax would have been a lot clearer if the designer of this class realized that the limits belong *between* the choices, such as

```
no houses|1|one house|2|{1} houses // not the actual format
```

You can use the `<` symbol to denote that a choice should be selected if the lower bound is strictly less than the value.

You can also use the ≤ symbol (U+2264) as a synonym for `#`. If you like, you can even use the infinity symbol (U+221E) to specify a lower bound of –∞ for the first value.

For example,

```
-∞<no houses|0<one house|2≤{1} houses
```

Let's finish our natural disaster scenario. If we put the choice string inside the original message string, we get the following format instruction:

```
String pattern = """
On {2,date,long}, {0} destroyed {1,choice,0#no houses|1#one house|2#{1}houses} \
and caused {3,number,currency} of damage.
""";
```

Or, in German,

```
String pattern = """
{0} zerstörte am {2,date,long} {1,choice,0#kein Haus|1#ein Haus|2#{1} Häuser} \
und richtete einen Schaden von {3,number,currency} an.
""";
```

Note that the ordering of the words is different in German, but the array of objects you pass to the `format` method is the *same*. The order of the placeholders in the format string takes care of the changes in the word ordering.

7.6. Text Boundaries

The `BreakIterator` class segments text into characters, words, sentences, or lines. The iteration protocol is not very pretty:

```
String s = "A man. A plan. Panama!";
BreakIterator iter = BreakIterator.getSentenceInstance(locale);
iter.setText(s);
int start = iter.first();
int end = iter.next();
while (end != BreakIterator.DONE) {
    String segment = s.substring(start, end);
    start = end;
    end = iter.next();
    IO.println(segment);
}
```

There are four iterators, returned by these static methods:

- The `getCharacterInstance` method yields an iterator over the grapheme clusters—that is, the Unicode sequences that produce a visible character.
- The `getWordInstance` method yields an iterator that produces either words or word separators. It is up to you to figure out the type of each segment.
- The iterator of the `getSentenceInstance` method segments the text into sentences.
- The `getLineInstance` iterator yields lines.

Each method has a version with no parameter, using the default locale, and one with a `Locale` parameter.



Tip: Instead of iterating over grapheme clusters with a `BreakIterator`, you can use a regular expression:

```
String[] clusters = s.split("\\b{g}");
```

7.7. Text Input and Output

As you know, the Java programming language itself is fully Unicode-based. However, Windows and Mac OS X still support legacy character encodings such as Windows-1252 or Mac Roman in Western European countries, or Big5 in Taiwan. Therefore, communicating with your users through text is not as simple as it should be. The following sections discuss the complications that you may encounter.

7.7.1. Text Files

Nowadays, it is best to use UTF-8 for saving and loading text files. But you may need to work with legacy files. If you know the expected character encoding, you can specify it when writing or reading text files:

```
Charset windows1252 = Charset.forName("Windows-1252");
var out = new PrintWriter(filename, windows1252);
```

For a guess of the user's preferred encoding, use the “native encoding” that is derived from the system executing the Java virtual machine. As of Java 18, its name is the value of the `native.encoding` system property.

Prior to Java 18, the native encoding is the default charset, which you can obtain from the static `Charset.defaultCharset()` method.

7.7.2. Line Endings

This isn't an issue of locales but of platforms. On Windows, text files are expected to use `\r\n` at the end of each line, where UNIX-based systems only require a `\n` character. Nowadays, most Windows programs can deal with just a `\n`, but older programs and the console may give your users grief.

Any line written with the `println` method will be properly terminated. The only problem is if you print strings that contain `\n` characters. They are not automatically modified to the platform line ending.

Instead of using `\n` in strings, you can use `printf` and the `%n` format specifier to produce platform-dependent line endings. For example,

```
out.printf("Hello%nWorld%n");
```

produces

```
Hello\r\nWorld\r\n
```

on Windows and

```
Hello\nWorld\n
```

everywhere else.

7.7.3. The Console

If you write programs that communicate with the user through `IO`, `System.in`/`System.out`, or `System.console()`, you have to face the possibility that the console may use a character encoding that is different from the default charset. This is particularly noticeable when working with the `cmd` shell on Windows. In the US version of Windows 11, the command shell still uses the archaic `IBM437` encoding that originated with IBM personal computers in 1982. The `Charset.defaultCharset()` method will return the `Windows-1252` character set, which is quite different. For example, the euro symbol € is present in `Windows-1252` but not in `IBM437`. When you call

```
IO.println("100 €");
```

the console will display

```
100 ?
```

Ideally, of course, your users should switch the console to UTF-8. In Windows, the command is

```
chcp 65001
```

As of Java 18, that is sufficient to make Java use UTF-8 in the console. In prior versions, it is also necessary to set the platform encoding with the `file.encoding` system property:

```
java -Dfile.encoding=UTF-8 MyProgram
```

If you cannot make your users change the character encoding of the console, you can find out what it is. As of Java 18, the system properties `stdout.encoding` and `stderr.encoding` give you that information. You can also change the settings on the command line.

You can also call `System.console().charset()`, provided that `System.console()` is not `null`.



Caution: If you obtain user input from a `new Scanner(System.in)`, the scanner uses the default charset and not the console character encoding. If they don't match, this will not work for keyboard input. If `System.console()` is not `null`, use a `new Scanner(System.console().reader())` instead.

7.7.4. The UTF-8 Byte Order Mark

As already mentioned, it is a good idea to use UTF-8 for text files when you can. However, if your application has to read UTF-8 text files created by other programs, you run into another potential problem. It is perfectly legal to add a “byte order mark” character U+FEFF as the first character of a file. In the UTF-16 encoding, where each code unit is a two-byte quantity, the byte order mark tells a reader whether the file uses “big-endian” or “little-endian” byte ordering. UTF-8 is a single-byte encoding, so there is no need to specify a byte ordering. But if a file starts with bytes `0xEF 0xBB 0xBF` (the UTF-8 encoding of U+FEFF), that is a strong indication that it uses UTF-8. For that reason, the Unicode standard encourages this practice. Any reader is supposed to discard an initial byte order mark.

There is just one fly in the ointment. The Oracle Java implementation stubbornly refuses to follow the Unicode standard, citing potential compatibility issues. That means that you, the programmer, must do what the platform won't do. When you read a text file and encounter a `U+FEFF` at the beginning, ignore it.



Caution: Unfortunately, the JDK implementors do not follow this advice. When you pass the `javac` compiler a valid UTF-8 source file that starts with a byte order mark, compilation fails with an error message "illegal character: \65279".

7.7.5. Character Encoding of Source Files

You, the programmer, will need to communicate with the Java compiler—and you do that with tools on your local system. For example, if you use a Chinese or Japanese version of Windows, your Java source code files may be encoded with a local character encoding. Such source code files are *not portable*. (Only the compiled class files are portable—they use a "modified UTF-8" encoding for identifiers and strings.)

Non-portable source files are not ideal. Nowadays it is easy to configure text editors and development environments to use UTF-8 for source files. As of Java 18, the `javac` compiler expects UTF-8 encoded source files by default. With earlier versions, run the compiler as:

```
javac -encoding UTF-8 MyProgram.java
```

To compile legacy source files that are not in UTF-8, specify the encoding in the same way.

7.8. Resource Bundles

When localizing an application, you'll probably have a dauntingly large number of message strings, button labels, and so on, that all need to be translated. To make this task feasible, you'll want to define the message strings in an external location, usually called a *resource*. The person carrying out the translation can then simply edit the resource files without having to touch the source code of the program.

In Java, you can use property files to specify string resources, and you can implement classes for resources of other types.



Note: Java technology resources are not the same as Windows or Macintosh resources. A Macintosh or Windows executable program stores resources, such as menus, dialog boxes, icons, and messages, in a section separate from the program code. A resource editor can inspect and update these resources without affecting the program code.



Note: Chapter 5 of Volume I describes a concept of JAR file resources, whereby data files, sounds, and images can be placed in a JAR file. The `getResource` method of the class `Class` finds the file, opens it, and returns a URL to the resource. By placing your files into the JAR file, you leave the job of finding the files to the class loader—which already knows how to locate items in a JAR file. However, that mechanism has no locale support.

7.8.1. Locating Resource Bundles

When localizing an application, you produce a set of *resource bundles*. Each bundle is a property file or a class that describes locale-specific items (such as messages, labels, and so on). For each bundle, you have to provide versions for all locales that you want to support.

You need to use a specific naming convention for these bundles. For example, resources specific to Germany go into a file `baseName_de_DE`, whereas those shared by all German-speaking countries go into `baseName_de`. In general, use

```
baseName_language_country
```

for all country-specific resources, and

```
baseName_language
```

for all language-specific resources. Finally, as a fallback, you can put defaults into a file without any suffix.

To load a bundle, use the command

```
ResourceBundle currentResources = ResourceBundle.getBundle(baseName, currentLocale);
```

The `getBundle` method attempts to load the bundle that matches the current locale by language and country. If it is not successful, the country and the language are dropped in turn. Then the same search is applied to the default locale, and, finally, the default bundle file is consulted. If even that attempt fails, the method throws a `MissingResourceException`.

That is, the `getBundle` method tries to load the following bundles:

```
baseName_currentLocaleLanguage_currentLocaleCountry
baseName_currentLocaleLanguage
baseName_defaultLocaleLanguage_defaultLocaleCountry
baseName_defaultLocaleLanguage
baseName
```

Once the `getBundle` method has located a bundle (say, `baseName_de_DE`), it will still keep looking for `baseName_de` and `baseName`. If these bundles exist, they become the *parents* of the `baseName_de_DE` bundle in a *resource hierarchy*. Later, when looking up a resource, the parents are searched if a lookup was not successful in the current bundle. That is, if a particular resource was not found in `baseName_de_DE`, then the `baseName_de` and `baseName` will be queried as well.

This is clearly a very useful service—and one that would be tedious to program by hand. The resource bundle mechanism of the Java programming language automatically locates the items that are the best match for a given locale. It is easy to add more and more localizations to an existing program—all you have to do is create additional resource bundles.



Note: If a locale has a script or variant, the lookup is quite a bit more complex. See the documentation of the method `ResourceBundle.Control.getCandidateLocales` for the gory details.



Tip: You need not place all resources for your application into a single bundle. You could have one bundle for button labels, one for error messages, and so on.

7.8.2. Property Files

Internationalizing strings is quite straightforward. You place all your strings into a property file such as `MyProgramStrings.properties`. This is simply a text file with one key/value pair per line. A typical file would look like this:

```
computeButton=Rechnen
colorName=black
defaultPaperSize=210×297
```

Then you name your property files as described in the preceding section, for example:

```
MyProgramStrings.properties
MyProgramStrings_en.properties
MyProgramStrings_de_DE.properties
```

You can load the bundle simply as

```
ResourceBundle bundle = ResourceBundle.getBundle("MyProgramStrings", locale);
```

To look up a specific string, call

```
String computeButtonLabel = bundle.getString("computeButton");
```

Property files are encoded as UTF-8.



Caution: Before Java 9, files for storing properties had to be ASCII files, and non-ASCII characters had to be encoded using the `\u`*xxxx* notation. For example, `"colorName=Grün"` had to be specified as:

```
colorName=Gr\u00FCn
```

7.8.3. Bundle Classes

To provide resources that are not strings, define classes that extend the `ResourceBundle` class. Use the standard naming convention to name your classes, for example

```
MyProgramResources.java
MyProgramResources_en.java
MyProgramResources_de_DE.java
```

Load the class with the same `getBundle` method that you use to load a property file:

```
ResourceBundle bundle = ResourceBundle.getBundle("MyProgramResources", locale);
```



Caution: When searching for bundles, a bundle in a class is given preference over a property file when the

two bundles have the same base names.

Each resource bundle class implements a lookup table. You need to provide a key string for each setting you want to localize, and use that key string to retrieve the setting. For example,

```
var backgroundColor = (Color) bundle.getObject("backgroundColor");
double[] paperSize = (double[]) bundle.getObject("defaultPaperSize");
```

The simplest way to implement resource bundle classes is to extend the `ListResourceBundle` class. The `ListResourceBundle` lets you place all your resources into an object array and then does the lookup for you. Follow this code outline:

```
public class baseName_language_country extends ListResourceBundle {
    private static final Object[][] contents = {
        { key1, value2 },
        { key2, value2 },
        . . .
    }
    public Object[][] getContents() { return contents; }
}
```

For example,

```
public class ProgramResources_de extends ListResourceBundle {
    private static final Object[][] contents = {
        { "backgroundColor", Color.black },
        { "defaultPaperSize", new double[] { 210, 297 } }
    }
    public Object[][] getContents() { return contents; }
}

public class ProgramResources_en_US extends ListResourceBundle {
    private static final Object[][] contents = {
        { "backgroundColor", Color.blue },
        { "defaultPaperSize", new double[] { 216, 279 } }
    }
    public Object[][] getContents() { return contents; }
}
```



Note: The paper sizes are given in millimeters. Everyone on the planet, with the exception of the United States and Canada, uses ISO 216 paper sizes. For more information, see **https://www.cl.cam.ac.uk/~mgk25/iso-paper.html**.

Alternatively, your resource bundle classes can extend the `ResourceBundle` class. Then you need to implement two methods, to enumerate all keys and to look up the value for a given key:

```
Enumeration<String> getKeys()
Object handleGetObject(String key)
```

The `getObject` method of the `ResourceBundle` class calls the `handleGetObject` method that you supply.

`java.util.ResourceBundle` 1.1

- `static ResourceBundle getBundle(String baseName, Locale locale)`
- `static ResourceBundle getBundle(String baseName)`
  load the resource bundle class with the given name, for the given locale or the default locale, and its parent classes. If the resource bundle classes are located in a package, the base name must contain the full package name, such as `"intl.ProgramResources"`. The resource bundle classes must be `public` so that the `getBundle` method can access them.
- `Object getObject(String name)`
  looks up an object from the resource bundle or its parents.
- `String getString(String name)`
  looks up an object from the resource bundle or its parents and casts it as a string.
- `String[] getStringArray(String name)`
  looks up an object from the resource bundle or its parents and casts it as a string array.
- `Enumeration<String> getKeys()`
  returns an enumeration object to enumerate the keys of this resource bundle. It enumerates the keys in the parent bundles as well.
- `Object handleGetObject(String key)`
  should be overridden to look up the resource value associated with the given key if you define your own resource lookup mechanism.

7.9. A Complete Example

In this section, we apply the material of this chapter to localize a retirement calculator. The program calculates whether or not you are saving enough money for your retirement. You enter your age, how much money you save every month, and so on (see **Figure 7.4**).

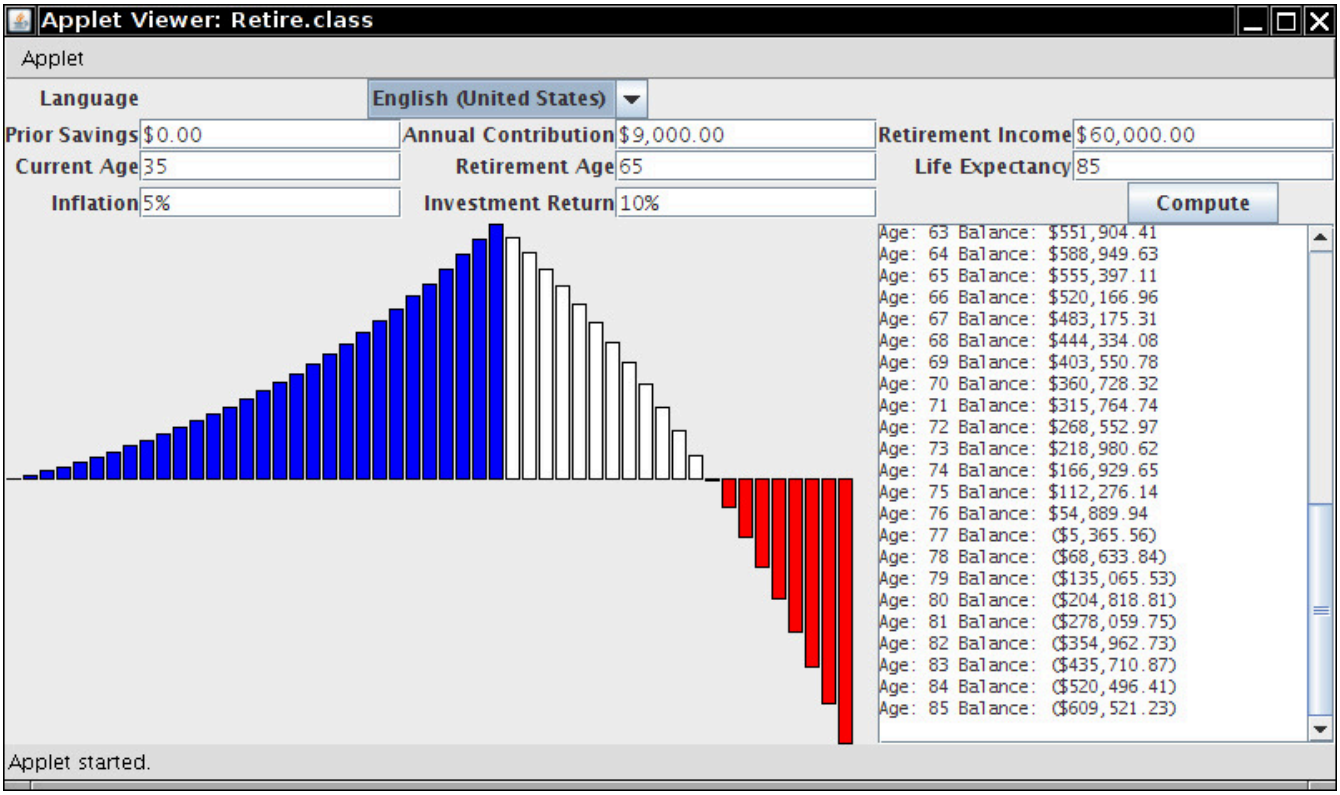


Figure 7.4: The retirement calculator in English

The text area and the graph show the balance of the retirement account for every year. If the numbers turn negative toward the later part of your life and the bars in the graph appear below the x axis, you need to do something

—for example, save more money, postpone your retirement, die earlier, or be younger.

The retirement calculator works in three locales (English, German, and Chinese). Here are some of the highlights of the internationalization:

- The labels, buttons, and messages are translated into German and Chinese. You can find them in the classes `RetireResources_de` and `RetireResources_zh`. English is used as the fallback—see the `RetireResources` file.
- Whenever the locale changes, we reset the labels and reformat the contents of the text fields.
- The text fields handle numbers, currency amounts, and percentages in the local format.
- The computation field uses a `MessageFormat`. The format string is stored in the resource bundle of each language.
- Just to show that it can be done, we use different colors for the bar graph, depending on the language chosen by the user.

Figure 7.5 shows the output in German. To see Chinese characters (as in Figure 7.6), be sure you have Chinese fonts installed and configured with your Java runtime. Otherwise, Chinese characters will show up as "missing character" icons.

Listing 7.4 shows the code of a simplified text version, sadly without colors. Listing 7.5 is the property file for the English strings. The localized strings are in Listing 7.6 (German) and Listing 7.7 (Chinese).

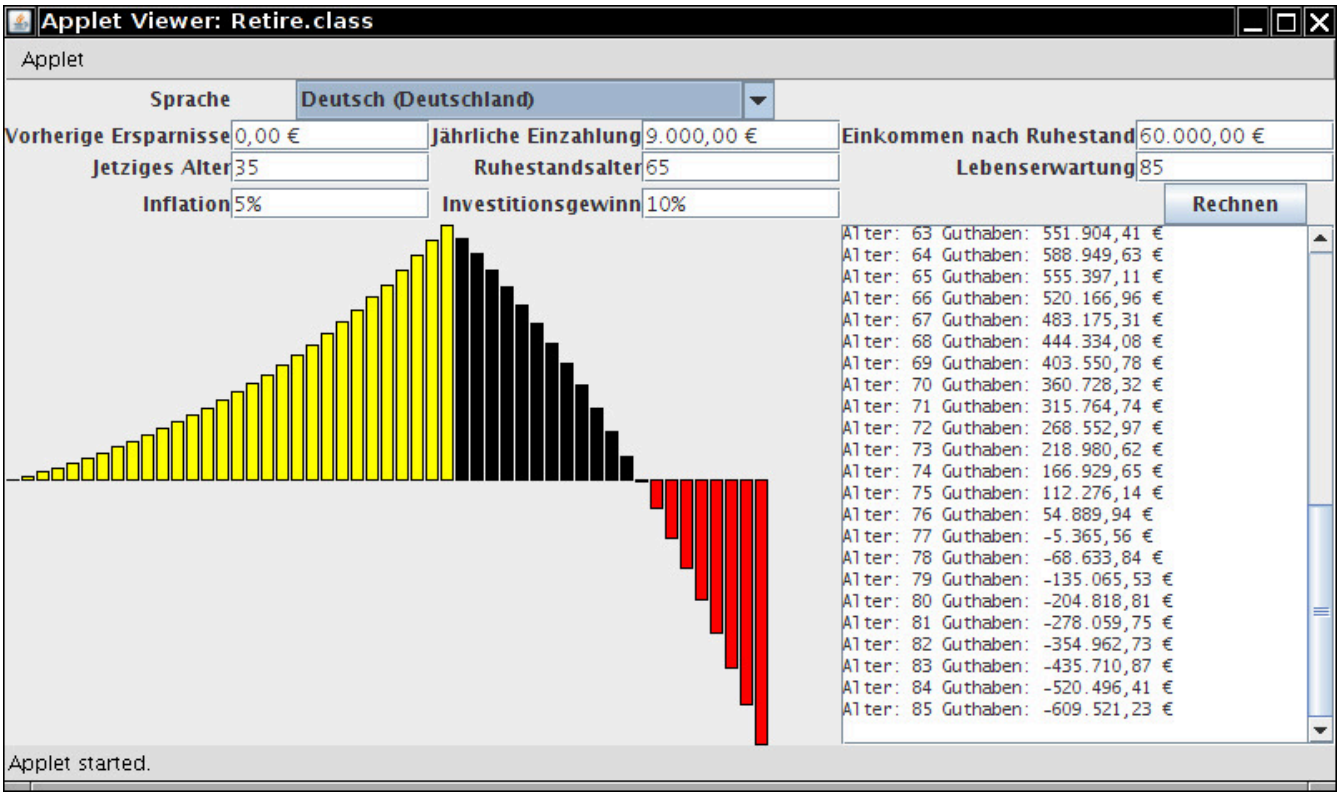


Figure 7.5: The retirement calculator in German

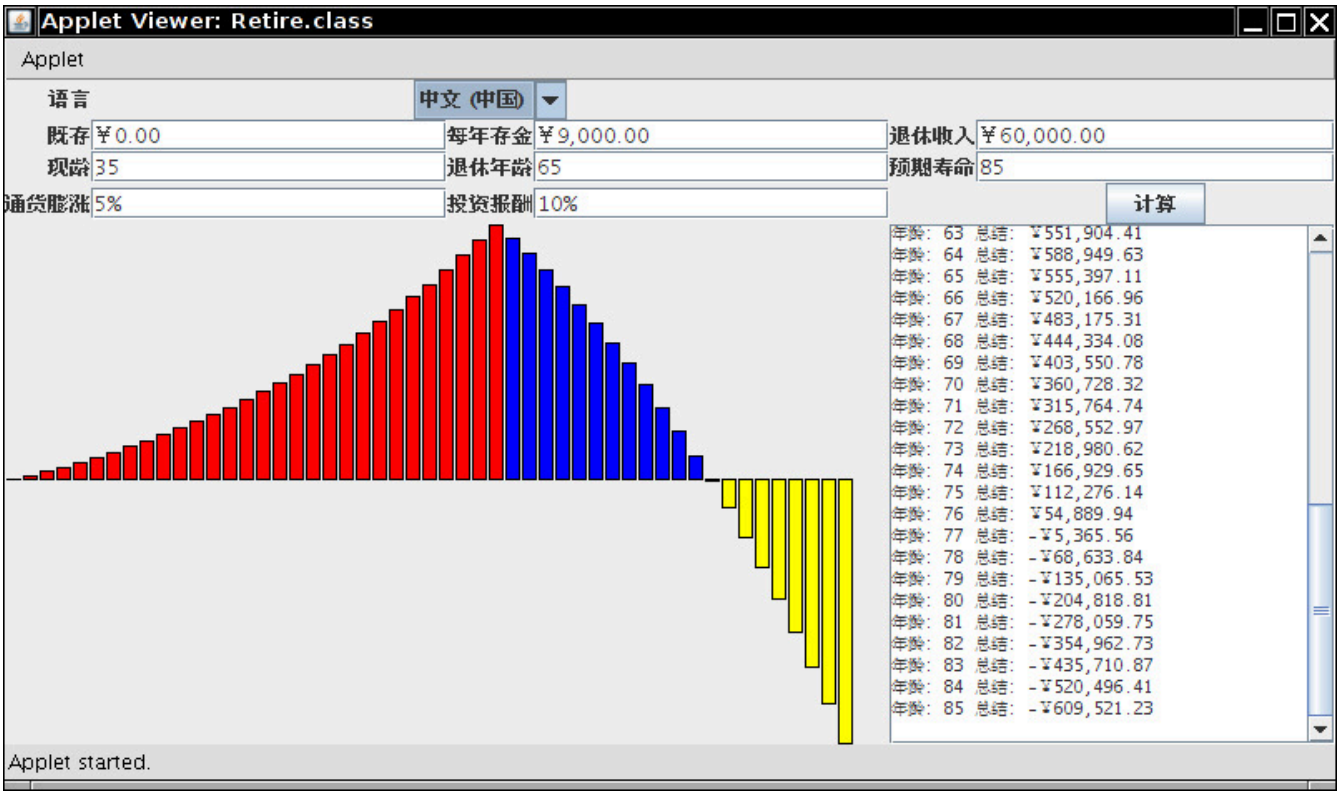


Figure 7.6: The retirement calculator in Chinese

Listing 7.4 v2ch07/retire2/Retire2.java

```
1  package v2ch07.retire2;
2
3  import module java.base;
4
5  import v2ch07.util.Choices;
6
7  /**
8   * This program shows a retirement calculator. The prompts are displayed in English, German, or
9   * Chinese.
10  */
11 class Retire2 {
12     void main() throws Exception {
13         // Get locale
14         ResourceBundle bundle = ResourceBundle.getBundle("v2ch07.retire.RetireStrings",
15                 Locale.getDefault());
16         IO.println(bundle.getString("language"));
17         Locale[] locales = { Locale.US, Locale.CHINA, Locale.GERMANY };
18         Locale currentLocale = Choices.choose(locales, Locale::getDisplayName);
19         bundle = ResourceBundle.getBundle("v2ch07.retire.RetireStrings", currentLocale);
20         NumberFormat integerFormat = NumberFormat.getIntegerInstance(currentLocale);
21         NumberFormat numberFormat = NumberFormat.getNumberInstance(currentLocale);
22
23         // Prompt for parameters
24         double savings = numberFormat.parse(
25                 IO.readln(bundle.getString("savings") + ": ")).doubleValue();
26         double contributions = numberFormat.parse(
27                 IO.readln(bundle.getString("contrib") + ": ")).doubleValue();
28         double income = numberFormat.parse(
29                 IO.readln(bundle.getString("income") + ": ")).doubleValue();
30         int currentAge = integerFormat.parse(
31                 IO.readln(bundle.getString("currentAge") + ": ")).intValue();
32         int retireAge = integerFormat.parse(
```

```
                IO.readln(bundle.getString("retireAge") + ": ")).intValue();
        int deathAge = integerFormat.parse(
                IO.readln(bundle.getString("deathAge") + ": ")).intValue();
        double inflationPercent = numberFormat.parse(
                IO.readln(bundle.getString("inflationPercent") + ": ")).doubleValue();
        double investPercent = numberFormat.parse(
                IO.readln(bundle.getString("investPercent") + ": ")).doubleValue();

        // Show values
        var retireMsg = new MessageFormat("");
        retireMsg.setLocale(currentLocale);
        retireMsg.applyPattern(bundle.getString("retire"));

        double balance = savings;
        for (int year = currentAge; year <= deathAge; year++) {
            Object[] msgArgs = { year, balance };
            IO.println(retireMsg.format(msgArgs));
            if (year < retireAge)
                balance += contributions;
            else
                balance -= income;
            balance = balance * (1 + (investPercent - inflationPercent));
        }
    }
}
```

Listing 7.5 v2ch07/retire2/RetireStrings.properties

```
language=Language
computeButton=Compute
savings=Prior Savings
contributions=Annual Contribution
income=Retirement Income
currentAge=Current Age
retireAge=Retirement Age
deathAge=Life Expectancy
inflationPercent=Inflation
investPercent=Investment Return
retire=Age: {0,number} Balance: {1,number,currency}
```

Listing 7.6 v2ch07/retire2/RetireStrings_de.properties

```
language=Sprache
computeButton=Rechnen
savings=Vorherige Ersparnisse
contributions=Jährliche Einzahlung
income=Einkommen nach Ruhestand
currentAge=Jetziges Alter
retireAge=Ruhestandsalter
deathAge=Lebenserwartung
inflationPercent=Inflation
investPercent=Investitionsgewinn
retire=Alter: {0,number} Guthaben: {1,number,currency}
```

Listing 7.7 v2ch07/retire2/RetireStrings_zh.properties

```
language=语言
computeButton=计算
savings=既存
contributions=每年存金
income=退休收入
currentAge=现龄
retireAge=退休年龄
deathAge=预期寿命
inflationPercent=通货膨涨
investPercent=投资报酬
retire=年龄: {0,number} 总结: {1,number,currency}
```

You have seen how to use the internationalization features of the Java language. You can now use resource bundles to provide translations into multiple languages, and use formatters and collators for locale-specific text processing.

In the next chapter, we will delve into scripting and compiling.

Previous chapter
< 6. The Date and Time API

Next chapter
8. Compiling and Scripting >